

GrosoVPN: Protocolo de Túnel Cifrado de Capa 3 (Implementación Propia)

Proyecto Final de Criptografía

Una implementación completa de una VPN (Virtual Private Network) construida **desde cero** en Python puro, sin utilizar librerías criptográficas externas (como OpenSSL, Cryptography o PyCryptodome). Este proyecto demuestra la implementación de primitivas criptográficas, diseño de protocolos seguros, manejo de redes a bajo nivel (TUN interfaces) y defensa contra ataques de red.

Descripción General

GrosoVPN establece un túnel virtual seguro sobre redes IP no confiables. Captura paquetes IP de la Capa 3 mediante una interfaz virtual (TUN), los encapsula, cifra y autentica, y los transporta mediante UDP.

Características Principales:

- **Zero-Dependency:** Toda la criptografía (ChaCha20, SHA-256, HMAC, HKDF, DH) está implementada nativamente.
- **Confidencialidad:** Cifrado de flujo de alta velocidad.
- **Integridad y Autenticidad:** Esquema *Encrypt-then-MAC* para prevenir modificaciones.
- **Forward Secrecy:** Intercambio de claves efímero (Ephemeral Diffie-Hellman) por sesión.
- **Anti-Replay:** Protección contra ataques de repetición mediante contadores de secuencia monótonos.
- **Resiliencia:** Capaz de transportar cualquier protocolo IP (ICMP, TCP, UDP) y tipos de archivo (PDF, ISO, etc.).

Arquitectura Criptográfica

Todas las primitivas se encuentran en el módulo `crypto/` y fueron escritas siguiendo los RFCs oficiales.

1. Cifrado Simétrico: ChaCha20 (RFC 7539)

Se seleccionó ChaCha20 sobre AES por su resistencia a ataques de canal lateral (timing attacks) en implementaciones por software y su alto rendimiento.

- **Estado:** Matriz de 4x4 palabras de 32 bits.
- **Operación:** ARX (Add-Rotate-Xor).
- **Keystream:** Generación de flujo pseudoaleatorio XOR-eado con el texto plano.

2. Integridad: HMAC-SHA256 (RFC 2104)

Para garantizar que el mensaje no ha sido alterado.

- **Hash Base:** SHA-256 (Implementación propia de la estructura Merkle-Damgård).
- **Construcción:** $H(K \parallel \text{opad} \parallel H(K \parallel \text{ipad} \parallel \text{msg}))$.

3. Acuerdo de Claves: Diffie-Hellman (RFC 3526)

- **Grupo:** MODP Group 14 (2048-bit prime).
- **Seguridad:** Basada en la dificultad del Problema del Logaritmo Discreto.
- **Propiedad:** Claves efímeras generadas en cada ejecución (PFS - Perfect Forward Secrecy).

4. Derivación de Claves: HKDF (RFC 5869)

El secreto compartido g^{ab} no se usa directamente. Se pasa por una función de derivación basada en HMAC para obtener material de clave criptográficamente fuerte.

- **Salida (76 bytes):**
 - Key_Enc (32 bytes): Para ChaCha20.
 - Key_Mac (32 bytes): Para HMAC.
 - Nonce_Base (12 bytes): IV inicial.

Diseño del Protocolo

1. Fase de Handshake (Autenticado)

Antes de enviar datos, Cliente y Servidor negocian claves de sesión.

1. **CLIENT_INIT:** [Tag: CLIENT] + [NonceC] + [PubKeyC]
2. **SERVER_RESP:** [Tag: SERVER] + [NonceS] + [PubKeyS] + [HMAC_PSK(Transcript)]
 - El servidor autentica la respuesta usando una Clave Pre-Compartida (PSK).
3. **CLIENT_FIN:** [Tag: CLIENT_FIN] + [HMAC_PSK(Transcript)]
 - El cliente verifica al servidor y prueba su propia identidad.

2. Fase de Transporte (Encrypt-then-MAC)

Cada paquete de red interceptado por la interfaz TUN se encapsula con la siguiente estructura para garantizar seguridad contra ataques de texto cifrado elegido (IND-CCA):

Header (4B)	Ciphertext (Variable)	Auth Tag (32B)
Counter (BE)	ChaCha20(Payload)	HMAC(Header + Ciphertext)

- **Counter:** Previene ataques de Replay (el receptor descarta contadores viejos).
- **Tag:** Autentica tanto el texto cifrado como el encabezado (Header).

Estructura del Proyecto

None

```
tunnel_project/  
├── groso1.py      # Cliente VPN (Punto final A)  
├── groso2.py      # Servidor VPN (Punto final B)
```

```

├─ crypto/                # LIBRERÍA CRIPTOGRÁFICA PROPIA
│   ├── __init__.py
│   ├── chacha20.py       # Cifrado de flujo
│   ├── sha256.py         # Función Hash
│   ├── hmac.py           # Código de autenticación
│   ├── hkdf.py           # Derivación de claves
│   ├── dh.py             # Diffie-Hellman (Math core)
│   └─ aead.py            # Composición Encrypt-then-MAC
├─ net/                   # MÓDULO DE RED
│   ├── handshake.py      # Lógica del protocolo de acuerdo
│   ├── tun_if.py         # Interacción con Kernel Linux (ioctl)
│   ├── client.py         # Bucle principal del Cliente
│   └─ server.py          # Bucle principal del Servidor
├─ demo_client.sh         # Script de automatización (Demo)
├─ demo_server.sh         # Script de automatización (Demo)
├─ pcap_decryptor.py      # Herramienta forense (Análisis de tráfico)
└─ decrypt_bin.py         # Herramienta de descifrado manual (Sin
dependencias)

```

Requisitos e Instalación

Este proyecto está diseñado para correr en sistemas **Linux** debido al uso de interfaces TUN/TAP.

- **Sistema Operativo:** Ubuntu 20.04+, Debian, Kali Linux (o WSL2 en Windows).
- **Python:** Versión 3.6 o superior.
- **Permisos:** Se requieren privilegios de root (sudo) para crear interfaces de red.

Instalación:

No es necesaria ninguna instalación (pip install prohibido para el núcleo). Simplemente clonar o copiar la carpeta.

(Opcional: Solo para la herramienta forense pcap_decryptor.py, se requiere scapy para leer archivos PCAP. El túnel funciona sin esto).

Guía de Uso

Escenario de Laboratorio

- **Groso2 (Servidor):** IP Física 192.168.10.12
- **Groso1 (Cliente):** IP Física 192.168.10.11

1. Iniciar el Servidor (Groso2)

Shell

```

sudo python3 groso2.py \
  --listen 0.0.0.0 \
  --port 50000 \
  --tun-ip 10.20.0.1/30 \

```

```
--tun-name tun0
```

2. Iniciar el Cliente (Groso1)

Shell

```
sudo python3 groso1.py \  
  --peer 192.168.10.12 \  
  --port 50000 \  
  --tun-ip 10.20.0.2/30 \  
  --tun-name tun0
```

3. Probar Conectividad

Desde el cliente, hacer ping a la IP virtual del servidor:

Shell

```
ping 10.20.0.1
```

4. Transferencia de Archivos

Utilizando netcat para enviar un archivo a través del túnel:

- **Servidor:** nc -u -l -p 8080 > recibido.pdf
- **Cliente:** cat documento.pdf | nc -u 10.20.0.1 8080

Herramientas Forenses y Validación

El proyecto incluye scripts para auditar la seguridad y demostrar el funcionamiento del cifrado.

decrypt_bin.py (Zero-Dependencies)

Descifra un payload binario exportado manualmente desde Wireshark.

Shell

```
python3 decrypt_bin.py paquete_exportado.bin  
# Introducir claves HEX cuando se soliciten.
```

pcap_decryptor.py (Análisis Masivo)

Toma una captura de tráfico .pcap cifrada y genera un nuevo archivo .pcap con todo el tráfico descifrado, permitiendo reconstruir sesiones completas (PDFs, imágenes, HTTP).

Shell

```
python3 pcap_decryptor.py captura_vpn.pcap captura_limpia.pcap
```

Nota: Esta herramienta utiliza scapy únicamente para el parseo de archivos PCAP. El descifrado se realiza con la librería interna crypto.

Análisis de Seguridad

Amenaza	Mitigación Implementada
Intercepción (Sniffing)	Cifrado ChaCha20. El atacante solo ve ruido aleatorio.
Modificación (Tampering)	HMAC-SHA256 en esquema Encrypt-then-MAC. Cualquier bit cambiado invalida el paquete.
Replay Attack	Contador de 32 bits en cabecera. El servidor descarta paquetes con Counter <= Last_Seen.
Man-in-the-Middle (MITM)	Autenticación mutua mediante PSK en el Handshake + Intercambio Diffie-Hellman.
Compromiso Futuro	Perfect Forward Secrecy (PFS). Las claves de sesión se borran de RAM al finalizar.

Autores

Autores: Ximena Ruiz Ramírez, Valeria Liv Larumbe Colmenares

Institución: Universidad Panamericana

Este software fue desarrollado con fines educativos para la demostración de competencias en criptografía aplicada y programación de redes.